Módulo 6 — Quantização e Otimização de Modelos

Estratégias de Eficiência para Deep Learning em Produção e Edge AI

Ementa e Objetivos

Teoria de Base

- Treinamento vs Inferência.
- Motivação técnica e econômica.
- Tipos de representação numérica.

Técnicas de Otimização

- Post-Training Quantization (PTQ).
- Quantization-Aware Training (QAT).
- Structured Pruning (Poda).

Implementação

- Frameworks (TFLite, ONNX).
- Benchmark de hardware.
- Análise de energia e latência.

Seção 1

A Lacuna de Hardware: Treino vs. Inferência

O Ciclo de Vida do Modelo

Treinamento (Cloud)

Objetivo: Ajuste dos pesos (aprendizado).

- Alta precisão (FP32).
- Gradientes necessários.
- Altíssimo consumo de RAM e energia.

Inferência (Edge)

Objetivo: Predição rápida e barata.

- Precisão reduzida (INT8).
- Apenas passo adiante (Forward).
- Restrições térmicas e de bateria.

O Desafio do Deploy

Um modelo de treinamento padrão como a ResNet-50 em FP32 exige aproximadamente 100MB de espaço e milhões de operações de ponto flutuante por imagem.

Hardware de Treino

NVIDIA A100: 80GB VRAM, 300W TDP.

Hardware de Inferência

ESP32: 520KB SRAM, Alimentado por pilha.

Sem otimização, a IA no Edge é impossível.

O que é Quantização?

É o mapeamento de um conjunto grande de valores (contínuos) para um conjunto menor de valores (discretos).

Em Deep Learning, trocamos o tipo de dado de alta precisão por um de menor ocupação em bits.

$$v = S \cdot (q - Z)$$

Onde v é o valor real, S é a escala, q o valor quantizado e Z o ponto zero.

Exemplo Prático:

Valor original (FP32): 0.347821474...

↓ Quantização

Valor mapeado (INT8): 44

Economia de 75% no armazenamento.

Orçamento de Bits

Formato	Tamanho	Precisão	Range Aproximado
Float32 (Single)	4 bytes	7 dígitos	±3.4e38
Float16 (Half)	2 bytes	3 dígitos	±65.504
BFloat16 (Brain)	2 bytes	2 dígitos	Mesmo do FP32
INT8 (Integer)	1 byte	Discreto	-128 a 127

Impacto da Precisão Reduzida

A perda de precisão introduz ruído nos pesos, o que chamamos de **Erro de Quantização**.

MAE (Erro Médio)

FP32 → FP16: 0.000172

FP32 → INT8: 0.003891

Erro Máximo

FP32 → FP16: 0.001953

FP32 → INT8: 0.009804

O segredo é que redes neurais são **roborentas ao ruído**, permitindo que a acurácia final quase não mude.

Benefícios da Quantização

Espaço

Modelos de 100MB caem para 25MB em INT8. Essencial para dispositivos com memória flash limitada.

Velocidade

Multiplicações inteiras são mais rápidas que as de ponto flutuante em quase todos os hardwares embarcados.

Energia

Menor tráfego de dados e operações mais simples resultam em até 4x mais autonomia de bateria.

Trade-offs de Performance

Critério	Float32	Float16	INT8
Acurácia	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐
Velocidade	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
Economia RAM	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
Economia Energia	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
Hardware Alvo	Server GPU	Mobile GPU/TPU	MCU/IoT

Seção 2

Estratégias de Quantização: PTQ vs. QAT

Post-Training Quantization (PTQ)

A quantização ocorre **após** o modelo já estar totalmente treinado.

- Fácil de aplicar.
- Não exige re-treinamento.
- Usa um pequeno "dataset de calibração" para estimar o range dos dados.

Vantagem Principal

Rapidez. Ideal para converter modelos prontos para o Edge em poucos minutos.

Quantization-Aware Training (QAT)

O modelo é treinado ou ajustado (fine-tuned) **simulando** os erros de quantização.

- Melhor acurácia possível em INT8.
- O modelo "aprende" a compensar o ruído da baixa precisão.
- Exige mais poder computacional e tempo.

Vantagem Principal

Qualidade. Reduz drasticamente a queda de acurácia em modelos sensíveis.

Pipeline TFLite (TensorFlow Lite)

O fluxo padrão da Google para IA embarcada:

1. Treinamento

Desenvolvimento do modelo (.keras) no PC ou Cloud.

2. Conversão

Uso do TFLite Converter para gerar o arquivo .tflite.

3. Deploy

Execução no dispositivo via TFLite Runtime.

Atividade Prática: Conversão TFLite

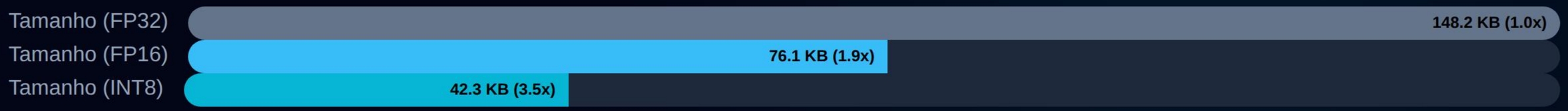
Como converter um modelo Keras para os três formatos no Python:

```
# 1. Float32 (Sem quantização)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_f32 = converter.convert()

# 2. Float16
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]
tflite_f16 = converter.convert()

# 3. INT8 (Post-Training)
converter.representative_dataset = calibration_gen
tflite_int8 = converter.convert()
```

Benchmark: Resultados Práticos



Dados baseados no modelo de reconhecimento de dígitos (MNIST) treinado no curso.

Benchmark: Velocidade de Inferência

Ganhos de 2x em velocidade com apenas 1.4% de perda em acurácia.

Consumo de Energia no Edge

7x

Mais predições

Estimativa para 1000 inferências (Cortex-M4):

- Float32: 42.0 mJ (~9.4M inferências por carga)
- INT8: 5.7 mJ (~69.3M inferências por carga)

A quantização é a diferença entre durar 1 mês ou 7 meses.

Seção 3

Structured Pruning: Poda de Redes Neurais

O que é Pruning (Poda)?

Se a quantização reduz a precisão de cada conexão, o **Pruning** remove as conexões que não são importantes.

Unstructured Pruning

Zera pesos individuais. O modelo continua do mesmo tamanho físico, mas fica cheio de zeros (esparso).

Structured Pruning

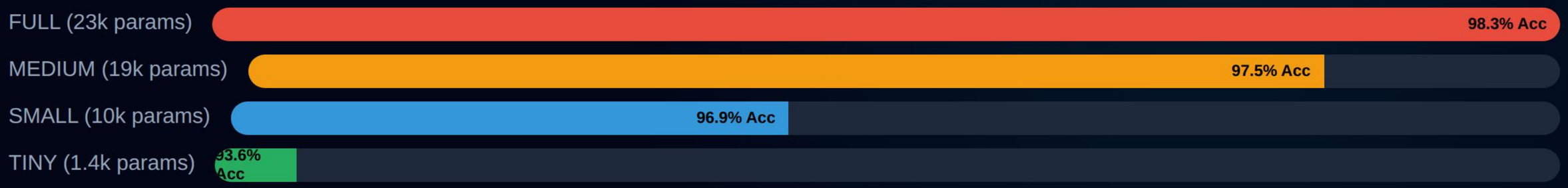
Remove canais de convolução ou neurônios inteiros. O modelo **realmente diminui** de tamanho.

Análise de Sensibilidade

Nem todas as camadas são iguais. Algumas podem ser podadas agressivamente, outras são vitais.

Camada	Queda Acc ao zerar	Peso L1 (Magnitude)	Ação Recomendada
Conv2D (Inicial)	+28.5%	Alta	Manter
Dense (Intermediária)	+3.1%	Baixa	**Podar (Eliminar)**
Output (Softmax)	+92.4%	Crítica	Nunca mexer

Jornada da Poda Estruturada



Podemos reduzir o número de parâmetros em **15 vezes** mantendo uma acurácia comercialmente viável (>90%).

Otimização Combinada

Poda Estruturada + Quantização INT8

6.8x Compressão Total

2.7x Speedup Total

Baseline (148KB) → Otimizado (21KB)

Fluxo de Decisão de Deploy

1. **Acurácia aceitável no Edge?** Se não, fine-tune ou QAT.
2. **Ainda grande ou lento?** Aplicar Pruning Estruturado.
3. **Restrição extrema de RAM?** Usar INT8 + TF Micro.
4. **Hardware possui NPU?** Usar formato específico (TensorRT/OpenVINO).

Seção 4

Exercícios e Conclusão

Exercícios de Fixação

1. Calibração PTQ

Mude o tamanho do dataset de calibração de 200 para 50. Como isso afeta o erro médio?

2. Poda Manual

Tente remover a primeira camada de convolução. Por que a queda de acurácia é tão alta?

3. Deploy Simulado

Calcule quantas inferências o seu modelo faria em uma bateria de smartphone (4000mAh).

Resumo do Módulo 6

- **Quantização** não é apenas compressão, é viabilidade de hardware.
- **Float16** é o melhor equilíbrio entre velocidade e qualidade para Mobile.
- **INT8** é obrigatório para microcontroladores (MCU).
- **Pruning** ataca a arquitetura, enquanto a quantização ataca os dados.
- A calibração do dataset no PTQ é a etapa mais crítica para evitar erros grosseiros.

Dúvidas?

A otimização é o que separa um brinquedo de laboratório de um produto real de mercado.

Módulo 6 — Inteligência Artificial Embarcada